

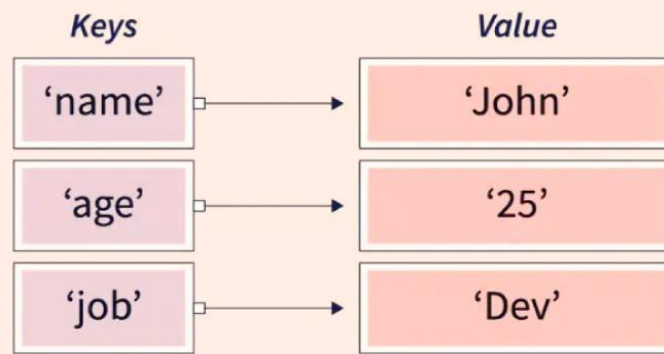
Map Interface Concepts with Function, Predicate, Producer, Consumer, and Supplier

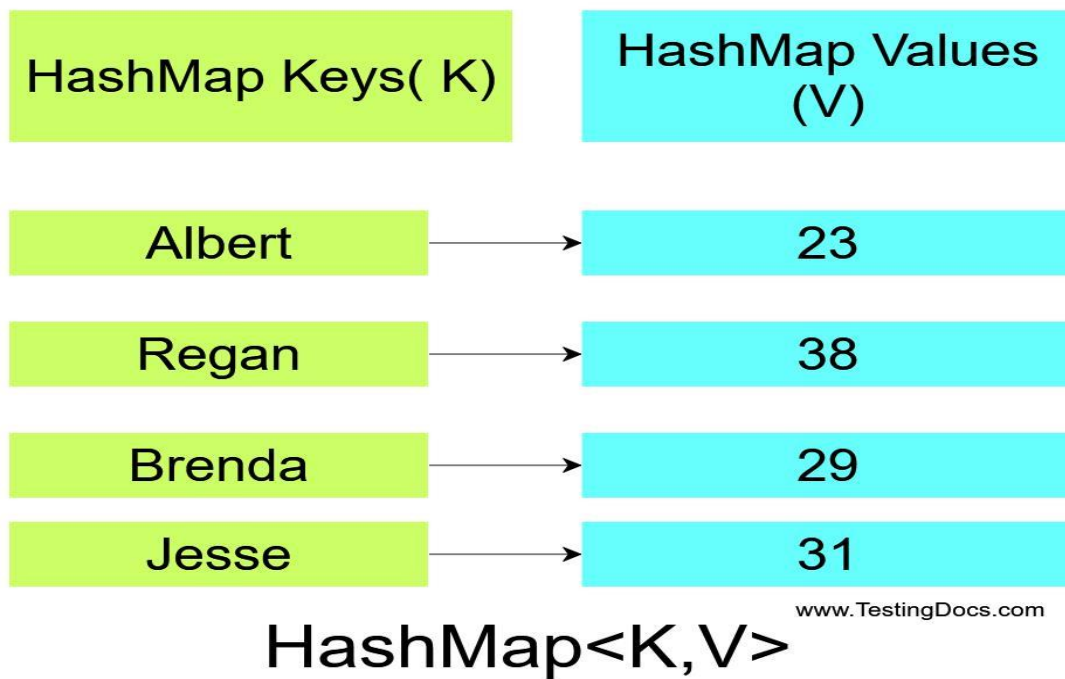
In Java, the **Map<K,V> interface** represents a collection of **key-value pairs**. When combined with Java 8+ functional interfaces such as Function, Predicate, Consumer, and Supplier, a map becomes very useful for **transforming, filtering, reading, updating, and generating data**.

Important distinction: **Map itself is not a functional interface**. It is a collection interface. Functional interfaces are used to *operate on the data stored in a Map*.

1. Basic Map Concept

What is Java Map?

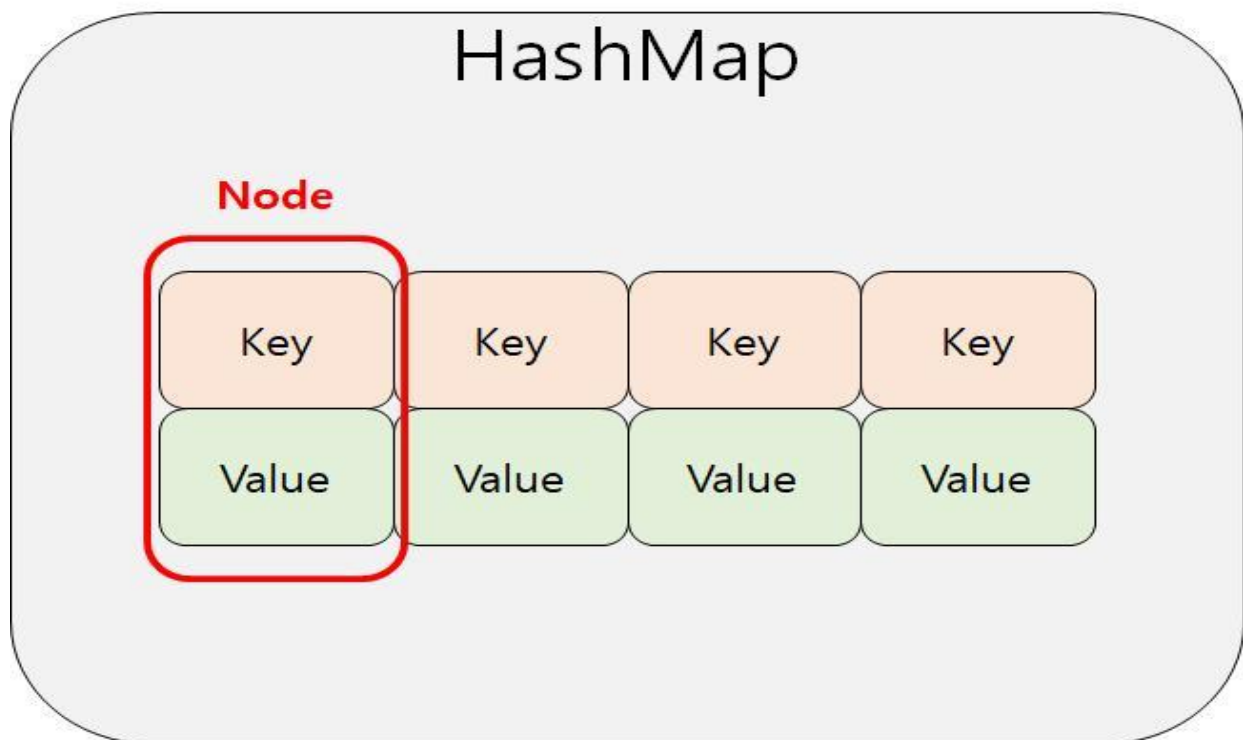
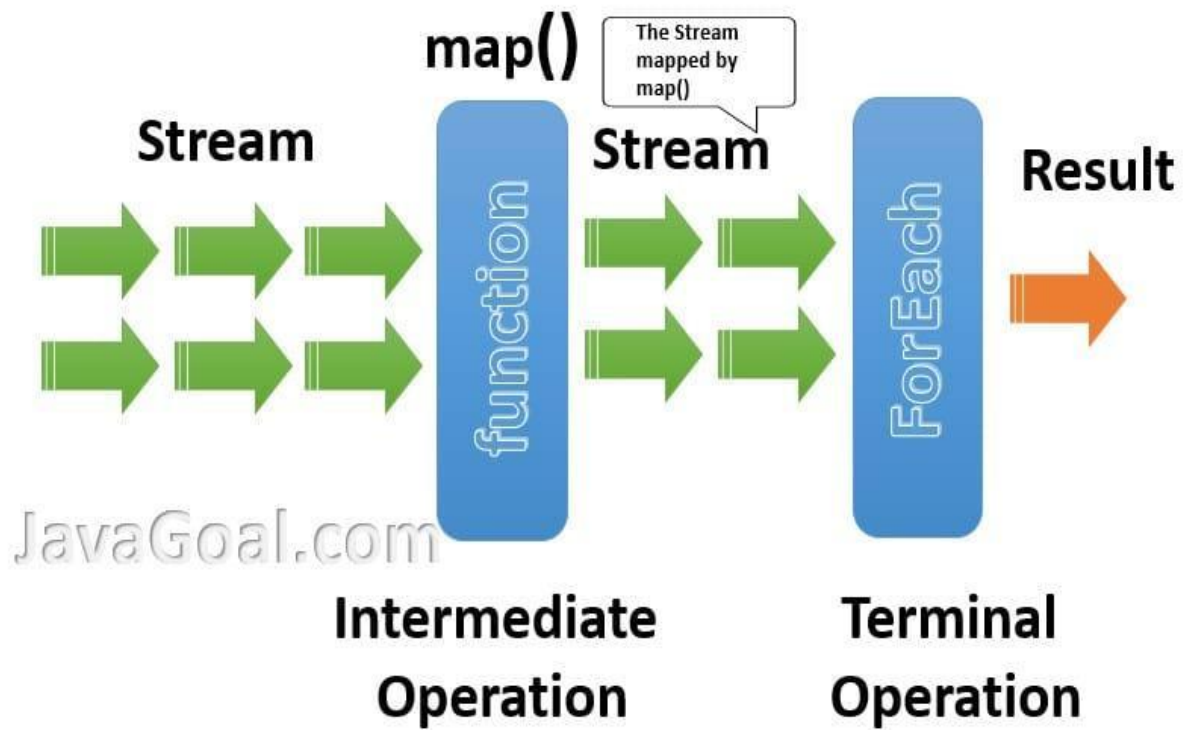




MASTERING FUNCTIONAL INTERFACES

PREDICATE | FUNCTION | CONSUMER | SUPPLIER





A Map<K,V> stores data like:

Key	Value
101	-> "Anurag"
102	-> "Rahul"
103	-> "Amit"

Example:

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");
```

Here:

K = Integer
V = String

Therefore:

Map<Integer, String>

means:

A map whose keys are Integer and whose values are String.

2. Map + Functional Interfaces — Overall Relationship

The easiest way to understand the relationship is:

Functional concept	Functional interface	Map usage
Transform	Function<T,R>	Convert key/value into another value
Test / Filter	Predicate<T>	Check keys or values
Produce	Function<T,R> / Supplier<R>	Generate values associated with keys
Consume	Consumer<T> / BiConsumer<T,U>	Read/process map entries

Supply	Supplier<T>	Provide default/generated values
Two inputs	BiFunction<T,U,R>	Transform key + value together
Two-input test	BiPredicate<T,U>	Test key + value together
Two-input consume	BiConsumer<T,U>	Consume key + value together

The most important Map methods related to functional programming are:

```

forEach()
replaceAll()
compute()
computeIfAbsent()
computeIfPresent()
merge()

```

These methods accept functional interfaces.

3. Map and Function<T,R>

Definition

Function<T,R> represents a function that:

T input → R output

Its main abstract method is:

```
R apply(T t);
```

Example:

```
Function<String, Integer> lengthFunction =
    str -> str.length();
```

Here:

String → Integer

4. Using Function with Map Values

Suppose:

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");
```

We can transform each value:

```
Function<String, String> upperCase =  
    name -> name.toUpperCase();  
  
employees.replaceAll(  
    (id, name) -> upperCase.apply(name)  
);
```

Result:

```
101 → ANURAG  
102 → RAHUL  
103 → AMIT
```

Execution

```
Map value  
↓  
Function<String,String>  
↓  
toUpperCase()  
↓  
new value
```

5. Map.replaceAll() and BiFunction

An important point:

replaceAll()

does **not** directly accept Function<T,R>.

Its signature is conceptually:

```
void replaceAll(BiFunction<? super K, ? super V, ? extends V> function)
```

Therefore, both the key and value can participate.

Example:

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 50000);  
salaries.put(102, 60000);  
salaries.put(103, 70000);
```

```
salaries.replaceAll(  
    (id, salary) -> salary + 5000  
);
```

Result:

```
101 → 55000  
102 → 65000  
103 → 75000
```

The conceptual operation is:

```
(K,V)  
↓  
BiFunction<K,V,V>  
↓  
new V
```

6. Map and Predicate

Definition

Predicate<T> represents a condition.

$T \rightarrow \text{boolean}$

Its main method is:

```
boolean test(T t);
```

Example:

```
Predicate<Integer> isGreaterThan50000 =  
    salary -> salary > 50000;
```

7. Predicate with Map

Suppose:

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 40000);
salaries.put(102, 60000);
salaries.put(103, 80000);
```

We can use a predicate to check values:

```
Predicate<Integer> highSalary =
    salary -> salary > 50000;

for (Integer salary : salaries.values()) {

    if (highSalary.test(salary)) {
        System.out.println(salary);
    }
}
```

Output:

```
60000
80000
```

The flow is:

```
Map values
↓
Predicate<Integer>
↓
test(value)
↓
true / false
```

8. Predicate + Map Entry

A Map.Entry<K,V> represents:

key + value

Therefore we can create:

```
Predicate<Map.Entry<Integer, Integer>> highSalary =
    entry -> entry.getValue() > 50000;
```

Then:

```
salaries.entrySet()
    .stream()
    .filter(highSalary)
```



```
.forEach(System.out::println);
```

This is a very common pattern.

Conceptually:

```
Map
↓
entrySet()
↓
Stream<Entry<K,V>>
↓
Predicate<Entry<K,V>>
↓
filter()
```

9. Predicate and removeIf()

Map does not directly provide removeIf().

But entrySet() is a collection, so we can use:

```
salaries.entrySet().removeIf(
    entry -> entry.getValue() < 50000
);
```

Example:

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 40000);
salaries.put(102, 60000);
salaries.put(103, 80000);
```

```
salaries.entrySet()
    .removeIf(entry -> entry.getValue() < 50000);
```

Result:

```
102 → 60000
103 → 80000
```

Here:

```
entry -> entry.getValue() < 50000
```

is a:

Predicate<Map.Entry<Integer,Integer>>

10. Map as a Producer

The word **producer** is important because it is closely related to the PECS principle:

Producer Extends, Consumer Super

But there are two different ideas that should not be confused:

Producer in Generics

A producer is something from which you **read/get values**.

Producer in Functional Programming

A producer is something that **generates/provides a value**.

For example:

Supplier<String>

is a functional producer.

11. Map as a Producer of Values

A map can produce:

map.get(key)

Example:

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");
```

```
String name = employees.get(101);
```

Here:

```
Key  
↓  
Map  
↓  
Value
```

The Map is acting as a **data source/producer**.

12. Supplier with Map

Definition

Supplier<T> supplies a value without receiving an input.

$() \rightarrow T$

Its abstract method is:

`T get();`

Example:

```
Supplier<String> defaultName =  
    () -> "Unknown";
```

Now consider:

```
Map<Integer, String> employees = new HashMap<>();
```

We can use a supplier to generate a default value.

For example:

```
Supplier<String> defaultName =  
    () -> "Unknown";
```

```
int id = 101;
```

```
String name = employees.getDefault(  
    id,  
    defaultName.get()  
);
```

Result:

Unknown

13. Important Difference: `getOrDefault()` vs `computeIfAbsent()`

This distinction is extremely important.

Consider:

```
Supplier<String> supplier =  
    () -> "Unknown";
```

With:

```
employees.getDefault(101, supplier.get());
```

the supplier is executed **before** `getDefault()` receives its argument.

With:

```
employees.computeIfAbsent(  
    101,  
    key -> "Unknown"  
);
```

the mapping function executes **only when the key is absent**.

So for lazy generation, `computeIfAbsent()` is usually more appropriate.

14. Map `computeIfAbsent()` — Producer Concept

Signature conceptually:

```
V computeIfAbsent(  
    K key,  
    Function<? super K, ? extends V> mappingFunction  
);
```

Notice:

`Function<K,V>`

is used.

Example:

```
Map<Integer, String> employees = new HashMap<>();  
  
employees.computeIfAbsent(  
    101,  
    id -> "Employee-" + id  
);
```

Result:

101 → Employee-101

The flow is:

Key
↓
Function<K,V>
↓
Value

So computeIfAbsent() is effectively:

Key → generate Value

15. Map as a Consumer

A **consumer** accepts data and performs an operation.

Consumer<T>:

T → void

Its method:

void accept(T t);

Example:

```
Consumer<String> printName =  
    name -> System.out.println(name);
```

16. Map forEach() — Consumer Concept

The Map method:

forEach()

accepts a BiConsumer<K,V>.

Conceptually:

```
void forEach(BiConsumer<? super K, ? super V> action)
```

Example:

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");
```

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " = " + name)  
);
```

Output:

```
101 = Anurag  
102 = Rahul  
103 = Amit
```

Here:

```
(id, name) -> System.out.println(...)
```

is:

```
BiConsumer<Integer, String>
```

17. Consumer vs BiConsumer with Map

This distinction is very important.

Consumer

```
Consumer<String>
```

takes one input:

String → void

Therefore:

```
employees.values()  
    .forEach(  
        name -> System.out.println(name)  
    );
```

Here name -> ... is:

```
Consumer<String>
```

BiConsumer

BiConsumer<Integer, String>

takes two inputs:

Integer + String → void

Therefore:

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " = " + name)  
);
```

Here:

(id, name) -> ...

is a:

BiConsumer<Integer,String>

18. Map + Supplier + Consumer

These can be combined.

Example:

```
Map<Integer, String> employees = new HashMap<>();
```

```
Supplier<Integer> idSupplier =  
    () -> 101;
```

```
Supplier<String> nameSupplier =  
    () -> "Anurag";
```

```
employees.put(  
    idSupplier.get(),  
    nameSupplier.get()  
);
```

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " = " + name)  
);
```

Conceptually:

Supplier
↓
produces data
↓
Map
↓
stores data
↓
BiConsumer
↓
processes data

19. Map + Function + Predicate + Consumer

This is one of the most useful combinations.

Suppose:

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 40000);  
salaries.put(102, 60000);  
salaries.put(103, 80000);
```

Function

Convert salary:

```
Function<Integer, Integer> increment =  
    salary -> salary + 5000;
```

Predicate

Check high salary:

```
Predicate<Integer> highSalary =  
    salary -> salary > 50000;
```

Consumer

Print salary:

```
Consumer<Integer> printer =  
    salary -> System.out.println(salary);
```


Now combine:

```
salaries.values()  
  .stream()  
  .filter(highSalary)  
  .map(increment)  
  .forEach(printer);
```

Execution:

```
Map  
↓  
values()  
↓  
Stream<Integer>  
↓  
Predicate  
↓  
filter  
↓  
Function  
↓  
map  
↓  
Consumer  
↓  
forEach
```

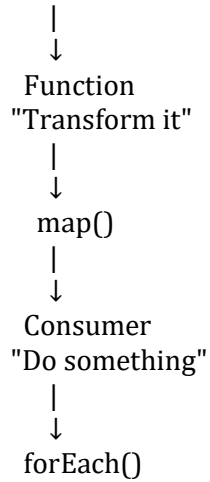
Result:

```
65000  
85000
```

20. Complete Functional Pipeline

A very useful mental model is:

```
MAP  
|  
↓  
source data  
|  
↓  
Predicate  
"Should I keep?"  
|  
↓  
filter()
```



For example:

```
employees.values()
    .stream()
    .filter(name -> name.length() > 5)
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Here:

Code	Functional concept
name -> name.length() > 5	Predicate
String::toUpperCase	Function
System.out::println	Consumer

21. Map + computeIfAbsent() — Function

This is one of the most important Map functional methods.

```
Map<Integer, String> employees = new HashMap<>();
```

```
String value =
    employees.computeIfAbsent(
        101,
        id -> "Employee-" + id
    );
```

The lambda:

id -> "Employee-" + id

is a:

Function<Integer,String>

because:

Integer → String

22. Map + computeIfPresent()

computeIfPresent() also accepts a BiFunction.

Example:

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 50000);
```

```
salaries.computeIfPresent(  
    101,  
    (id, salary) -> salary + 10000  
);
```

Result:

101 → 60000

The lambda:

(id, salary) -> salary + 10000

is:

BiFunction<Integer,Integer,Integer>

Flow:

Key + Existing Value

↓

BiFunction

↓

New Value

23. Map + compute()

Example:

```
salaries.compute(
    101,
    (id, salary) -> salary == null
        ? 50000
        : salary + 5000
);
```

Here:

(id, salary) -> ...

is a BiFunction.

It receives:

K + V

and returns:

V

24. Map + merge()

merge() is another important functional Map method.

Example:

```
Map<String, Integer> wordCount = new HashMap<>();
```

```
wordCount.put("Java", 1);
```

```
wordCount.merge(
    "Java",
    1,
    (oldValue, newValue) ->
        oldValue + newValue
);
```

Result:

Java → 2

The lambda:

```
(oldValue, newValue) -> oldValue + newValue
```

is:

```
BiFunction<Integer,Integer,Integer>
```

25. Real-World Example — Employee Map

Let's combine everything.

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");  
employees.put(104, "Suresh");
```

Predicate — find names longer than 5 characters

```
Predicate<String> longName =  
    name -> name.length() > 5;
```

Function — convert to uppercase

```
Function<String, String> upperCase =  
    String::toUpperCase;
```

Consumer — print

```
Consumer<String> printer =  
    System.out::println;
```

Pipeline

```
employees.values()  
    .stream()  
    .filter(longName)  
    .map(upperCase)  
    .forEach(printer);
```

Output:

```
ANURAG  
SURESH
```

26. Map.Entry Is Especially Important

When working with Map and functional programming, remember:

`Map.Entry<K,V>`

represents one complete key-value pair.

For example:

`101 → Anurag`

is represented conceptually as:

`Map.Entry<Integer, String>`

Therefore:

`employees.entrySet()`

provides:

`Stream<Map.Entry<K,V>>`

when converted to a stream.

This allows us to use:

`Predicate<Map.Entry<K,V>>`
`Function<Map.Entry<K,V>,R>`
`Consumer<Map.Entry<K,V>>`

27. Entry + Predicate + Function + Consumer

Example:

```
employees.entrySet()
    .stream()
    .filter(entry ->
        entry.getKey() > 101)
    .map(entry ->
        entry.getValue().toUpperCase())
    .forEach(name ->
        System.out.println(name));
```

Execution:

```
Map.Entry
↓
Predicate<Entry>
↓
```

filter
↓
Function<Entry,String>
↓
map
↓
Consumer<String>
↓
forEach

28. Producer, Consumer and PECS with Map

Now connect this to **PECS**.

Producer

If you only need to **read** data:

? extends T

Example:

List<? extends Number>

You can safely read Number values.

Consumer

If you need to **write/add** values:

? super T

Example:

List<? super Integer>

You can add Integer.

Map

For a Map:

Map<? extends K, ? extends V>

is generally viewed as a source from which you read key/value types.

While:

`Map<? super K, ? super V>`

can accept broader types where appropriate.

However, **Map's actual functional method signatures are more precise than simply saying "Map is a producer or consumer."**

For example:

```
computeIfAbsent(  
    K key,  
    Function<? super K, ? extends V> mappingFunction  
)
```

Notice the PECS pattern:

```
Function<? super K, ? extends V>  
    ↑      ↑  
Consumer  Producer
```

The function:

- **consumes** a K
- **produces** a V

This is a very important generics + functional-interface concept.

29. Most Important Map Functional Methods

Map method	Functional interface	Input	Output
forEach()	<code>BiConsumer<K,V></code>	<code>K + V</code>	<code>void</code>
replaceAll()	<code>BiFunction<K,V,V></code>	<code>K + V</code>	<code>V</code>
compute()	<code>BiFunction<K,V,V></code>	<code>K + V</code>	<code>V</code>
computeIfAbsent()	<code>Function<K,V></code>	<code>K</code>	<code>V</code>
computeIfPresent()	<code>BiFunction<K,V,V></code>	<code>K + V</code>	<code>V</code>
merge()	<code>BiFunction<V,V,V></code>	<code>old V + new V</code>	<code>V</code>

get()	—	K	V
getOrDefault()	—	K + V	V
put()	—	K + V	V
containsKey()	—	K	boolean
containsValue()	—	V	boolean

30. Function vs Predicate vs Consumer vs Supplier with Map

This table is the key to remembering everything.

Interface	Shape	Meaning	Typical Map usage
Function<T,R>	$T \rightarrow R$	Transform	computeIfAbsent()
Predicate<T>	$T \rightarrow \text{boolean}$	Test	filter(), removeIf()
Consumer<T>	$T \rightarrow \text{void}$	Process	values().forEach()
Supplier<T>	$() \rightarrow T$	Produce	Generate default data
BiFunction<T,U,R>	$T + U \rightarrow R$	Transform two inputs	compute(), merge(), replaceAll()
BiPredicate<T,U>	$T + U \rightarrow \text{boolean}$	Test two inputs	Map entry conditions
BiConsumer<T,U>	$T + U \rightarrow \text{void}$	Process two inputs	Map.forEach()

31. The Four Fundamental Questions

When looking at a Map operation, ask four questions:

1. Do I need to transform something?

Use:

Function<T,R>

Example:

name -> name.toUpperCase()

2. Do I need to check something?

Use:

Predicate<T>

Example:

name -> name.length() > 5

3. Do I need to process something?

Use:

Consumer<T>

Example:

name -> System.out.println(name)

4. Do I need to generate/provide something?

Use:

Supplier<T>

Example:

() -> "Unknown"

32. A Complete Example

```
import java.util.*;
```

```
import java.util.function.*;
```

```
public class MapFunctionalExample {
```

```
    public static void main(String[] args) {
```

```
        Map<Integer, String> employees = new HashMap<>();
```

```
        employees.put(101, "Anurag");
```

```

employees.put(102, "Rahul");
employees.put(103, "Suresh");
employees.put(104, "Amit");

// Predicate
Predicate<String> longName =
    name -> name.length() > 5;

// Function
Function<String, String> upperCase =
    String::toUpperCase;

// Consumer
Consumer<String> printer =
    System.out::println;

employees.values()
    .stream()
    .filter(longName)
    .map(upperCase)
    .forEach(printer);
}
}

```

Output:

ANURAG
SURESH

The complete algorithm is:

```

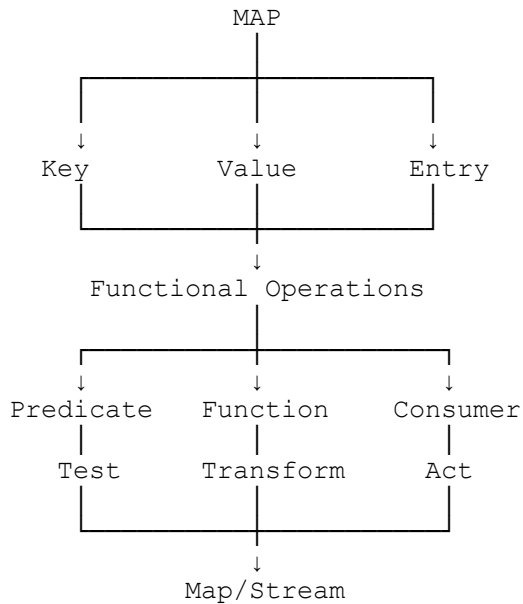
HashMap
↓
values()
↓
Stream
↓
Predicate
"Is name > 5?"
↓
filter()
↓
Function
"Convert uppercase"
↓
map()
↓
Consumer
"Print it"

```

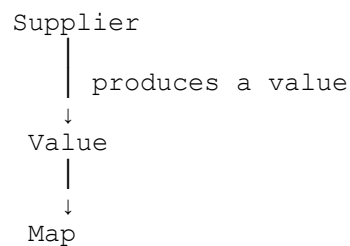
↓
forEach()

33. The Most Important Mental Model

You can remember the entire relationship like this:



And `Supplier` operates slightly differently:



So the simplest memory rule is:

Predicate	→ CHECK
Function	→ TRANSFORM
Consumer	→ PROCESS
Supplier	→ PRODUCE
Map	→ STORE / LOOKUP / ORGANIZE

And when two Map components are involved:

BiPredicate → CHECK two values

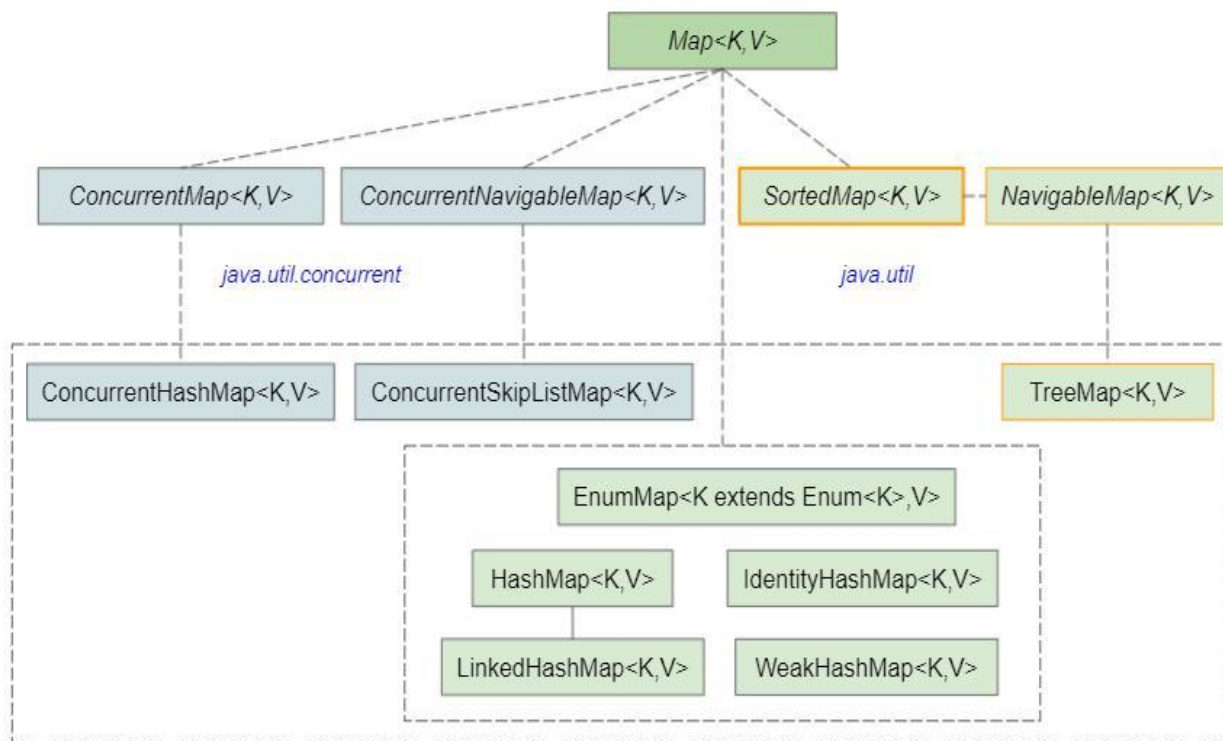
BiFunction → TRANSFORM two values
BiConsumer → PROCESS two values

The particularly important Map methods to master are **forEach()**, **replaceAll()**, **compute()**, **computeIfAbsent()**, **computeIfPresent()**, and **merge()**, because they show exactly how Java's Map API integrates functional interfaces with generics and PECS.

Yes. The important point is that **Map and its implementing classes do not themselves implement Predicate, Function, Consumer, or Supplier**. Rather, many Map APIs **accept functional interfaces as arguments**, allowing you to test, transform, consume, or produce map data.

Below is a detailed map of the major Java Map interfaces/classes and their relationship with functional programming.

1. Big Picture

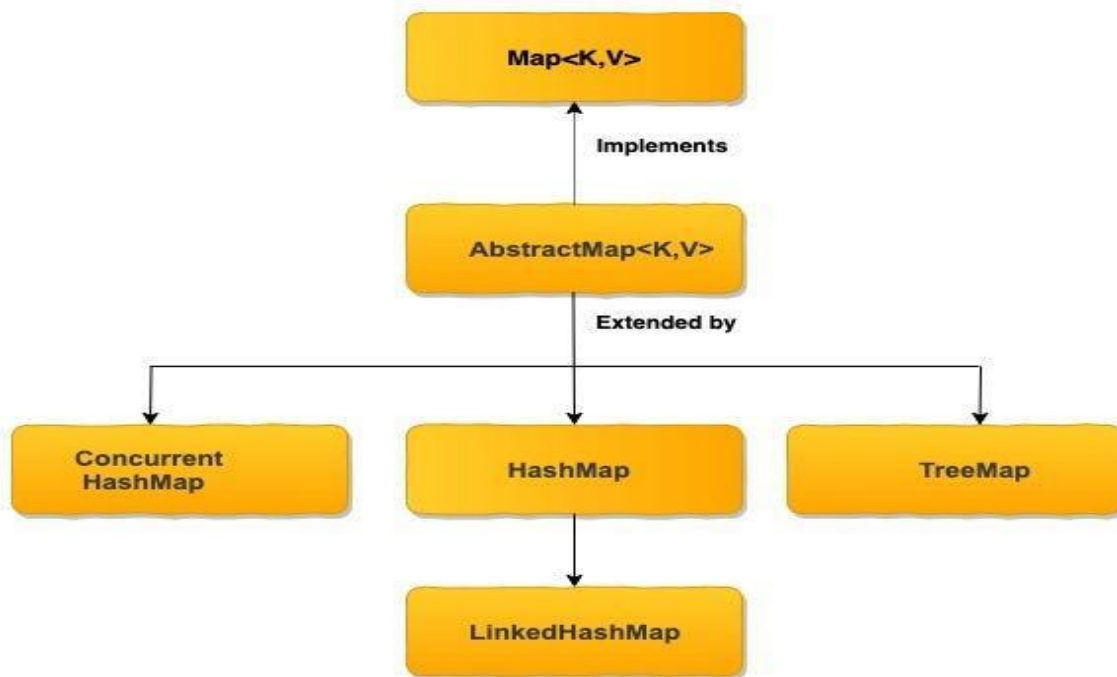


Built-in functional interfaces

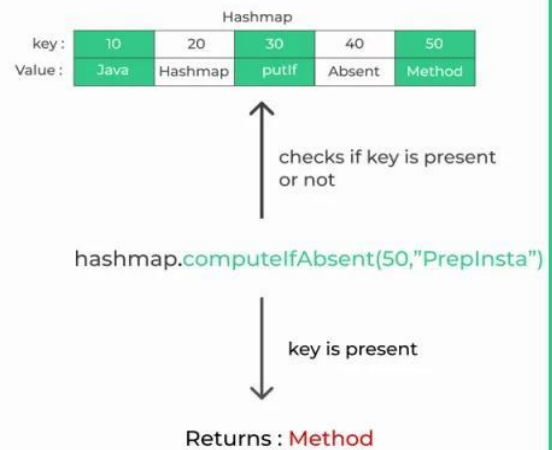
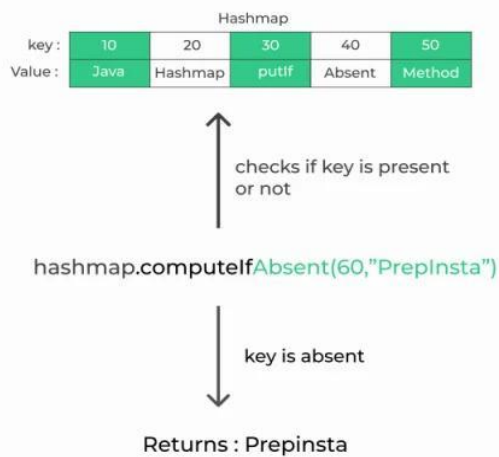
Predicate<T>	• boolean test(T t)
Consumer<T>	• void accept(T t)
Function<T, R>	• R apply(T t)
Supplier<T>	• T get()

Java Map Update Methods Cheat Sheet

Method	When Called	Parameter Type	Creates Value Lazily?	Return Value	Typical Use Case
<code>put(K key, V value)</code>	Always	Pre-computed value	✗ No	Previous value or <code>null</code>	Always overwrite (insert or replace)
<code>putIfAbsent(K key, V value)</code>	Only if key is absent or mapped to null	Pre-computed value	✗ No	Existing value or <code>null</code>	Insert default if not present
<code>computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)</code>	Only if key is absent or mapped to null	Function <code>(K -> V)</code>	✓ Yes	Existing or computed value	Lazily initialize complex objects
<code>computeIfPresent(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)</code>	Only if key is present and non-null	Function <code>(K, V) -> V</code>	✓ Yes	New value or <code>null</code> (removes entry if null)	Update existing entries
<code>merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)</code>	Always if key is present; inserts if absent	Pre-computed value + function <code>(oldValue, newValue) -> mergedValue</code>	✗ for value, ✓ for function	New value after merge	Combine counts, concatenate strings



Java Hashmap computeIfAbsent() Method

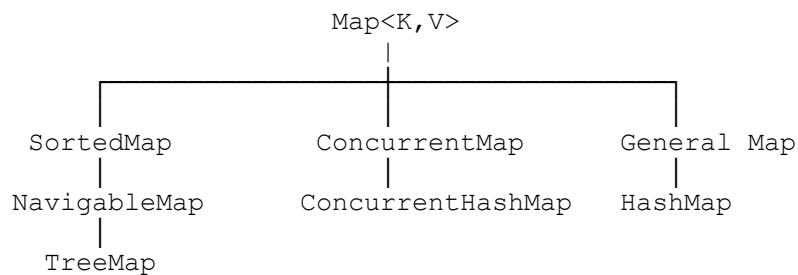


```

Map<Integer, Integer> map = new HashMap<>();
for (int num : nums) {
    map.merge(num, 1, Integer::sum);
}

```

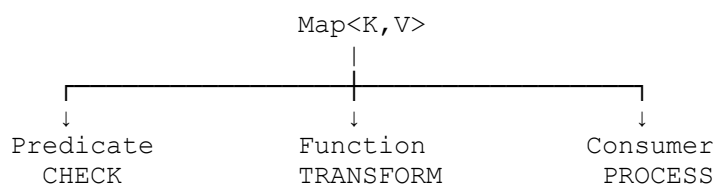
The hierarchy can be viewed approximately as:

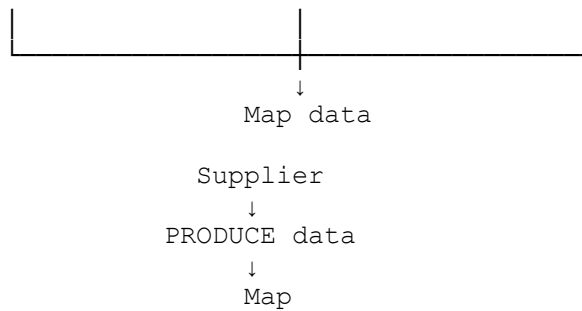


Other Map implementations:

- LinkedHashMap
- Hashtable
- WeakHashMap
- IdentityHashMap
- EnumMap

And functional programming sits **around** these Map implementations:





2. Four Functional Concepts

Before studying every Map implementation, remember these four basic functional interfaces.

Interface	Shape	Purpose
Predicate<T>	$T \rightarrow \text{boolean}$	Check/test
Function<T,R>	$T \rightarrow R$	Transform
Consumer<T>	$T \rightarrow \text{void}$	Process/use
Supplier<T>	$() \rightarrow T$	Produce/provide

There are also corresponding two-argument versions:

Interface	Shape
BiPredicate<T,U>	$(T,U) \rightarrow \text{boolean}$
BiFunction<T,U,R>	$(T,U) \rightarrow R$
BiConsumer<T,U>	$(T,U) \rightarrow \text{void}$

There is **no standard BiSupplier**, because a supplier doesn't consume an input.

3. Map's Most Important Functional Methods

These methods are inherited by most ordinary Map implementations:

```
forEach()
replaceAll()
compute()
computeIfAbsent()
computeIfPresent()
merge()
```

Their functional interfaces are:

Map method	Functional interface
forEach()	BiConsumer<K,V>
replaceAll()	BiFunction<K,V,V>
compute()	BiFunction<K,V,V>
computeIfAbsent()	Function<K,V>
computeIfPresent()	BiFunction<K,V,V>
merge()	BiFunction<V,V,V>

This is the central relationship.

4. HashMap + Functional Interfaces

HashMap<K,V> is the most commonly used general-purpose Map implementation.

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Amit");
```

4.1 HashMap.forEach() → Consumer

Because a Map has both key and value, it uses BiConsumer:

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " = " + name)  
);
```

Conceptually:

```
Integer + String  
↓  
BiConsumer<Integer,String>  
↓  
void
```

4.2 HashMap.computeIfAbsent() → Function

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.computeIfAbsent(  
    101,  
    id -> "Employee-" + id  
);
```

The lambda:

```
id -> "Employee-" + id
```

is:

```
Function<Integer, String>
```

because:

```
Integer → String
```

Result:

```
101 → Employee-101
```

4.3 HashMap.computeIfPresent() → BiFunction

```
Map<Integer, Integer> salaries = new HashMap<>();
```

```
salaries.put(101, 50000);
```

```
salaries.computeIfPresent(  
    101,  
    (id, salary) -> salary + 5000  
);
```

Here:

```
Integer + Integer
```

↓

```
BiFunction<Integer,Integer,Integer>
```

↓

```
Integer
```

Result:

```
101 → 55000
```

4.4 HashMap.merge() → BiFunction

```
Map<String, Integer> counts = new HashMap<>();

counts.put("Java", 1);

counts.merge(
    "Java",
    1,
    (oldValue, newValue) -> oldValue + newValue
);
```

Result:

Java → 2

The lambda is:

```
BiFunction<Integer, Integer, Integer>
```

5. LinkedHashMap + Functional Interfaces

LinkedHashMap maintains predictable iteration order, normally insertion order.

```
Map<Integer, String> employees =
    new LinkedHashMap<>();

employees.put(101, "Anurag");
employees.put(102, "Rahul");
employees.put(103, "Amit");
```

The functional methods work essentially the same way.

forEach() → BiConsumer

```
employees.forEach(
    (id, name) ->
        System.out.println(id + " : " + name)
);
```

computeIfAbsent() → Function

```
employees.computeIfAbsent(
    104,
    id -> "Suresh"
);
```

replaceAll() → BiFunction

```
employees.replaceAll(  
    (id, name) -> name.toUpperCase()  
);
```

The important distinction is:

HashMap

↓

Functional operations

LinkedHashMap

↓

Same functional operations

↓

But predictable iteration order

The functional interface relationship comes from the **Map contract**, not from LinkedHashMap specifically.

6. TreeMap + Functional Interfaces

TreeMap<K,V> stores keys in sorted order.

```
Map<Integer, String> employees =  
    new TreeMap<>();
```

```
employees.put(103, "Amit");  
employees.put(101, "Anurag");  
employees.put(102, "Rahul");
```

Iteration gives keys in sorted order:

101 → Anurag

102 → Rahul

103 → Amit

It supports the same major functional operations.

forEach() → BiConsumer

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " : " + name)  
);
```

computeIfAbsent() → Function

```
employees.computeIfAbsent(  
    104,  
    id -> "Suresh"  
);
```

replaceAll() → BiFunction

```
employees.replaceAll(  
    (id, name) -> name.toUpperCase()  
);
```

Important

TreeMap additionally provides navigation methods such as:

```
firstKey()  
lastKey()  
lowerKey()  
higherKey()  
floorKey()  
ceilingKey()  
subMap()  
headMap()  
tailMap()
```

These are not primarily functional-interface methods.

7. Hashtable + Functional Interfaces

Hashtable<K,V> is an older synchronized Map implementation.

```
Map<Integer, String> employees =  
    new Hashtable<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");
```

Modern Java versions provide the functional Map methods inherited from Map.

For example:

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " : " + name)  
);
```

And:

```
employees.replaceAll(  
    (id, name) -> name.toUpperCase()  
);
```

Important distinction

Hashtable is legacy code. For most new concurrent applications, ConcurrentHashMap is generally the more appropriate Map implementation.

8. WeakHashMap + Functional Interfaces

WeakHashMap<K,V> uses weak references for its keys.

Example:

```
Map<Object, String> map =  
    new WeakHashMap<>();
```

```
Object key = new Object();
```

```
map.put(key, "Some Data");
```

You can still use:

```
map.forEach(  
    (k, v) ->  
        System.out.println(k + " = " + v)  
);
```

And:

```
map.computeIfAbsent(  
    key,  
    k -> "Generated Data"  
);
```

The functional relationship remains:

```
WeakHashMap  
↓  
Map methods  
↓  
Function / BiFunction / BiConsumer
```

The special feature of WeakHashMap is **key garbage-collection behavior**, not functional programming.

9. IdentityHashMap + Functional Interfaces

IdentityHashMap<K,V> compares keys using **reference identity (==)**, rather than normal equals() semantics.

Example:

```
Map<String, Integer> map =  
    new IdentityHashMap<>();
```

```
String a = new String("Java");  
String b = new String("Java");
```

```
map.put(a, 100);  
map.put(b, 200);
```

a and b are different objects, so they can be different keys.

It also supports:

```
map.forEach(  
    (key, value) ->  
        System.out.println(key + " = " + value)  
);
```

and:

```
map.replaceAll(  
    (key, value) -> value + 10  
);
```

Again, the functional methods originate from the Map API.

10. EnumMap + Functional Interfaces

EnumMap<K,V> is optimized specifically for enum keys.

Example:

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY
```



```

}

Map<Day, String> schedules =
    new EnumMap<>(Day.class);

schedules.put(Day.MONDAY, "Java");
schedules.put(Day.TUESDAY, "Spring");

```

Functional operation:

```

schedules.forEach(
    (day, subject) ->
        System.out.println(day + " = " + subject)
);

```

computeIfAbsent():

```

schedules.computeIfAbsent(
    Day.WEDNESDAY,
    day -> "Database"
);

```

Here:

Day → String

is:

Function<Day, String>

11. ConcurrentHashMap + Functional Interfaces

This is particularly important in multithreaded applications.

```

ConcurrentHashMap<Integer, Integer> salaries =
    new ConcurrentHashMap<>();

salaries.put(101, 50000);

```

Then:

```

salaries.computeIfAbsent(
    102,
    id -> 60000
);

```

Or:

```
salaries.compute(  
    101,  
    (id, salary) -> salary + 5000  
);
```

Or:

```
salaries.merge(  
    101,  
    1000,  
    (oldValue, newValue) ->  
        oldValue + newValue  
);
```

And:

```
salaries.forEach(  
    (id, salary) ->  
        System.out.println(id + " = " + salary)  
);
```

ConcurrentHashMap is particularly useful because these Map operations have concurrency-aware semantics.

12. ConcurrentMap Interface

ConcurrentMap<K,V> extends Map<K,V> and provides concurrency-oriented Map behavior.

For example:

```
ConcurrentMap<String, Integer> map =  
    new ConcurrentHashMap<>();
```

It supports:

```
map.computeIfAbsent(  
    "Java",  
    key -> 1  
);
```

This is extremely useful for counters and caches.

For example:

```
map.merge(  
    "Java",
```

```
1,  
Integer::sum  
);
```

Repeated calls can maintain a count:

```
Java → 1  
Java → 2  
Java → 3  
Java → 4
```

13. SortedMap Interface

SortedMap<K,V> defines a sorted Map abstraction.

A common implementation is:

TreeMap

Example:

```
SortedMap<Integer, String> map =  
    new TreeMap<>();
```

The functional Map methods remain available:

```
map.forEach(  
    (id, name) ->  
        System.out.println(id + " = " + name)  
);
```

The special purpose of SortedMap is ordering/navigation, not functional interfaces.

14. NavigableMap Interface

NavigableMap<K,V> extends SortedMap.

Common implementation:

TreeMap

It provides methods such as:

```
lowerKey()  
floorKey()
```

```
ceilingKey()
higherKey()
pollFirstEntry()
pollLastEntry()
descendingMap()
```

Example:

```
NavigableMap<Integer, String> map =
    new TreeMap<>();
```

```
map.put(100, "A");
map.put(200, "B");
map.put(300, "C");
```

Then:

```
map.forEach(
    (key, value) ->
        System.out.println(key + " = " + value)
);
```

Again:

```
NavigableMap
  ↓
Map functional methods
  ↓
BiConsumer / BiFunction / Function
```

15. Predicate with Map Implementations

Here is an important point:

Map itself does not have a direct filter(Predicate) method.

Unlike Stream, you don't normally write:

```
map.filter(...)
```

Instead, use `entrySet()`, `keySet()`, or `values()`.

For example:

```
Map<Integer, Integer> salaries =
    new HashMap<>();
```

```
salaries.put(101, 40000);
```

```
salaries.put(102, 60000);
salaries.put(103, 80000);
```

Create:

```
Predicate<Map.Entry<Integer, Integer>> highSalary =
    entry -> entry.getValue() > 50000;
```

Then:

```
salaries.entrySet()
    .stream()
    .filter(highSalary)
    .forEach(System.out::println);
```

16. Predicate + removeIf()

You can directly remove entries using a predicate through the entry collection:

```
salaries.entrySet()
    .removeIf(
        entry -> entry.getValue() < 50000
    );
```

Conceptually:

```
Map
↓
entrySet()
↓
Predicate<Entry<K,V>>
↓
true → remove
false → keep
```

This works with Map implementations whose entry-set view supports the removal operation.

17. Function with All Map Implementations

The most important Map method using Function directly is:

```
computeIfAbsent()
```

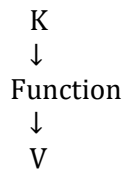
Example:

```
map.computeIfAbsent(  
    key,  
    k -> createValue(k)  
);
```

The lambda is:

Function<? super K, ? extends V>

Conceptually:



This means:

Given a key, calculate/create a value.

18. BiFunction with Map

Map heavily uses BiFunction.

compute()

```
map.compute(  
    key,  
    (k, v) -> newValue  
);
```

computeIfPresent()

```
map.computeIfPresent(  
    key,  
    (k, v) -> newValue  
);
```

replaceAll()

```
map.replaceAll(  
    (k, v) -> newValue  
);
```

merge()

```
map.merge(  
    key,  
    value,  
    (oldValue, newValue) -> result  
);
```

So:

```
BiFunction  
↓  
receives two values  
↓  
returns a value
```

19. Consumer with Map

Map.forEach() is the classic example.

```
map.forEach(  
    (key, value) ->  
        System.out.println(key + " = " + value)  
);
```

This is:

BiConsumer<K,V>

because:

```
K + V  
↓  
BiConsumer  
↓  
void
```

20. Supplier with Map

This requires special attention.

Map does **not** generally have methods such as:

```
get(Supplier<V>)
```

or:

```
put(Supplier<K>, Supplier<V>)
```

Instead, you can use a Supplier to create a value and then put it into the Map.

Example:

```
Supplier<String> employeeSupplier =  
    () -> "Anurag";
```

```
Map<Integer, String> employees =  
    new HashMap<>();
```

```
employees.put(  
    101,  
    employeeSupplier.get()  
);
```

Flow:

```
Supplier  
↓  
get()  
↓  
String  
↓  
Map.put()
```

21. Supplier + computeIfAbsent()

You can combine Supplier and Map operations.

For example:

```
Supplier<String> defaultEmployee =  
    () -> "Unknown";
```

```
employees.computeIfAbsent(  
    101,  
    key -> defaultEmployee.get()  
);
```

Here there are **two functional interfaces**:

```
key -> defaultEmployee.get()  
  ↑  
  Supplier
```

The outer lambda is a:

```
Function<Integer,String>
```


The supplier is:

Supplier<String>

This distinction is important.

22. Function vs Supplier in computeIfAbsent()

Compare:

Function

```
map.computeIfAbsent(  
    101,  
    id -> "Employee-" + id  
);
```

The generated value depends on the key:

```
key  
↓  
Function  
↓  
value
```

Supplier

```
Supplier<String> supplier =  
    () -> "Unknown";
```

```
map.computeIfAbsent(  
    101,  
    id -> supplier.get()  
);
```

The value does not depend on the key:

```
Supplier  
↓  
constant/generated value
```

The Map method still requires a Function, so the supplier has to be invoked inside that function.

23. Map + Stream + Predicate + Function + Consumer

This is probably the most important real-world combination.

```
Map<Integer, String> employees =  
    new HashMap<>();
```

```
employees.put(101, "Anurag");  
employees.put(102, "Rahul");  
employees.put(103, "Suresh");  
employees.put(104, "Amit");
```

Predicate

```
Predicate<Map.Entry<Integer, String>> employeeFilter =  
    entry -> entry.getValue().length() > 5;
```

Function

```
Function<Map.Entry<Integer, String>, String> nameConverter =  
    entry -> entry.getValue().toUpperCase();
```

Consumer

```
Consumer<String> printer =  
    System.out::println;
```

Complete pipeline

```
employees.entrySet()  
    .stream()  
    .filter(employeeFilter)  
    .map(nameConverter)  
    .forEach(printer);
```

Execution:

```
Map  
↓  
entrySet()  
↓  
Stream<Entry<K,V>>  
↓  
Predicate  
↓  
filter()  
↓  
Function  
↓
```

map()
↓
Consumer
↓
forEach()

24. Complete Example Using All Four

```
import java.util.*;
import java.util.function.*;

public class MapFunctionalDemo {

    public static void main(String[] args) {

        Map<Integer, String> employees =
            new HashMap<>();

        // Supplier
        Supplier<String> defaultName =
            () -> "Unknown";

        employees.put(101, "Anurag");
        employees.put(102, "Rahul");
        employees.put(103, "Suresh");

        // Function
        Function<String, String> upperCase =
            String::toUpperCase;

        // Predicate
        Predicate<String> longName =
            name -> name.length() > 5;

        // Consumer
        Consumer<String> printer =
            System.out::println;

        employees.values()
            .stream()
            .filter(longName)
            .map(upperCase)
            .forEach(printer);

        // Supplier used to create a value
        employees.put(104, defaultName.get());
    }
}
```

Result from the stream:

ANURAG
SURESH

25. Functional Interface Mapping Across Map Classes

The following table is the most useful summary.

Map interface/class	Predicate	Function	Consumer	Supplier
Map	Through views/Streams	computeIf Absent()	forEach()	External/helper usage
HashMap	Through views/Streams	Yes	Yes	External/helper usage
LinkedHashMap	Through views/Streams	Yes	Yes	External/helper usage
TreeMap	Through views/Streams	Yes	Yes	External/helper usage
Hashtable	Through views/Streams	Yes	Yes	External/helper usage
WeakHashMap	Through views/Streams	Yes	Yes	External/helper usage
IdentityHashMap	Through views/Streams	Yes	Yes	External/helper usage
EnumMap	Through views/Streams	Yes	Yes	External/helper usage
ConcurrentHashMap	Through views/Streams	Yes	Yes	External/helper usage
SortedMap	Through views/Streams	Yes	Yes	External/helper usage
NavigableMap	Through views/Streams	Yes	Yes	External/helper usage
ConcurrentMap	Through views/Streams	Yes	Yes	External/helper usage

The phrase "**Yes**" means the implementation inherits/supports the corresponding Map methods; it does **not** mean the Map class implements the functional interface.

26. Why Predicate Is Different

This is a particularly important conceptual distinction.

Map has:

```
forEach()  
compute()  
computeIfAbsent()  
computeIfPresent()  
merge()  
replaceAll()
```

but it does **not** have:

```
filter(Predicate)
```

Why?

Because a Map contains **key-value pairs**, while Predicate only describes a test.

Therefore you usually obtain a view first:

```
map.entrySet()  
map.keySet()  
map.values()
```

and then use:

```
stream().filter(predicate)
```

or:

```
entrySet().removeIf(predicate)
```

So:

```
Map  
↓  
entrySet()  
↓  
Predicate  
↓
```

filter/removelf

27. Why Consumer Is Directly Used by Map

A Map naturally needs an operation to process every key-value pair.

Therefore:

```
map.forEach(  
  (key, value) -> ...  
);
```

is naturally represented by:

`BiConsumer<K,V>`

Because:

```
Map.Entry  
↓  
key + value  
↓  
BiConsumer  
↓  
action
```

No value needs to be returned.

28. Why Function Is Used by Map

Maps often need to calculate a value based on a key.

For example:

```
map.computeIfAbsent(  
  key,  
  k -> createValue(k)  
);
```

This is naturally:

$K \rightarrow V$

which is exactly:

Function<K,V>

29. Why BiFunction Is Used So Often

Sometimes the calculation requires both key and existing value:

```
map.compute(  
    key,  
    (k, oldValue) -> newValue  
);
```

Therefore:

$K + V \rightarrow V$

requires:

BiFunction<K,V,V>

Likewise:

merge()

needs two values:

$oldValue + newValue \rightarrow mergedValue$

so it uses:

BiFunction<V,V,V>

30. PECS Connection

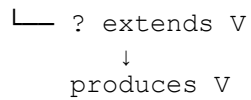
This becomes especially interesting when generics are included.

For example, computeIfAbsent() conceptually uses:

Function<? super K, ? extends V>

Read this as:

```
Function  
|  
├── ? super K  
|   ↓  
|   consumes K
```



This is the PECS principle:

Consumer Super, Producer Extends

So:

Function<? super K, ? extends V>

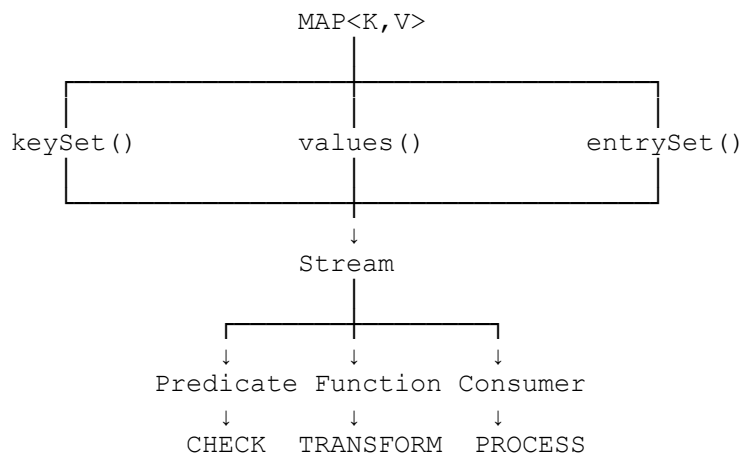
can be understood as:

Consumes K
Produces V

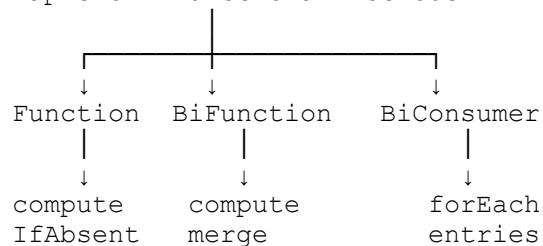
This is one of the best examples of **functional interfaces + generics + PECS + Map** working together.

31. Final Conceptual Architecture

You can remember the complete architecture like this:



Map's own functional methods



|
replaceAll

And Supplier sits outside the Map's core functional method set:

```
Supplier<T>
  |
  | get()
  ↓
  T value
  |
  ↓
Map.put() / Map operation
```

32. The Most Important Rules to Memorize

Rule 1 — Map is not a Functional Interface

Map<K,V>

is a collection/data-structure interface.

Rule 2 — Predicate means CHECK

Predicate<T>
 $T \rightarrow \text{boolean}$

Typical Map usage:

```
entrySet()
  .stream()
  .filter(...)
```

or:

```
entrySet().removeIf(...)
```

Rule 3 — Function means TRANSFORM

Function<T,R>
 $T \rightarrow R$

Typical Map usage:

```
computeIfAbsent()
```

Rule 4 — BiFunction means TRANSFORM using two inputs

`BiFunction<T,U,R>`

Typical Map methods:

`compute()`
`computeIfPresent()`
`merge()`
`replaceAll()`

Rule 5 — Consumer means PROCESS

`Consumer<T>`

Typical Map usage:

`map.values().forEach(...)`

Rule 6 — BiConsumer means PROCESS key + value

`BiConsumer<K,V>`

Typical Map usage:

`map.forEach(...)`

Rule 7 — Supplier means PRODUCE

`Supplier<T>`
`() → T`

A Supplier is generally **used with a Map**, rather than being a core functional parameter of the standard Map methods.

One-line memory formula

Predicate	= CHECK
Function	= TRANSFORM
BiFunction	= TRANSFORM using 2 inputs
Consumer	= PROCESS

BiConsumer	= PROCESS 2 inputs
Supplier	= PRODUCE
Map	= STORE + LOOKUP + ORGANIZE key/value data

And the most important Map-functional mapping is:

forEach()	→ BiConsumer
replaceAll()	→ BiFunction
compute()	→ BiFunction
computeIfAbsent()	→ Function
computeIfPresent()	→ BiFunction
merge()	→ BiFunction

entrySet()

↓

Stream

↓

filter()	→ Predicate
map()	→ Function
forEach()	→ Consumer

That last pipeline is the key connection between the **Map framework and Java's functional programming model**.